



main.c

```
83 -     printf("%c%c",
84 -         key[r1][(c1 + 1) % 5],
85 -         key[r2][(c2 + 1) % 5]);
86 - }
87 - else if(c1 == c2) // Same column
88 - {
89 -     printf("%c%c",
90 -         key[(r1 + 1) % 5][c1],
91 -         key[(r2 + 1) % 5][c2]);
92 - }
93 - else // Rectangle rule
94 - {
95 -     printf("%c%c",
96 -         key[r1][c2],
97 -         key[r2][c1]);
98 - }
99 - }
100 -
101 - printf("\n");
102 -
103 - return 0;
104 - }
```



Share

Run

Output

Clear

Enter plaintext: Must see you over Cadogan West. Coming at once.

Cipher Text: UZTBDLZRWQNPZLGBETQALOZATBTQFKQLTHPODG

=== Code Execution Successful ===



main.c



Share

Run

Output

Clear

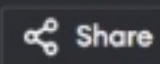
```
1 #include <stdio.h>
2 #include <math.h>
3
4 int main()
5 {
6     double log2Keys;
7
8     // log2(25!)
9     log2Keys = 83.68;
10
11     printf("Approximate number of possible Playfair keys = 2^%.2f\n", log2Keys);
12
13     return 0;
14 }
```

Approximate number of possible Playfair keys = 2^83.68

=== Code Execution Successful ===



main.c



Output

Clear

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  int main() {
6      int key[2][2] = {{9,4},{5,7}};
7      char text[200], plain[200];
8      int i, j = 0;
9
10     printf("Enter plaintext: ");
11     fgets(text, sizeof(text), stdin);
12
13     // Remove spaces and convert to uppercase
14     for(i = 0; text[i] != '\0'; i++) {
15         if(isalpha(text[i])) {
16             plain[j++] = toupper(text[i]);
17         }
18     }
19
20     // Add X if odd length
21     if(j % 2 != 0)
22         plain[j++] = 'X';
```

Enter plaintext: MEET ME AT THE USUAL PLACE AT TEN RATHER THAN EIGHT OCLOCK

Cipher Text: UKIXUKYDROMEIWSZXWIOKUNUKHXHROAJROANQYEBTLKJEGAD

=== Code Execution Successful ===



main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2
3 int modInverse(int a, int m)
4 {
5     a = a % m;
6     for(int x = 1; x < m; x++)
7         if((a * x) % m == 1)
8             return x;
9     return -1;
10 }
11
12 int main()
13 {
14     // Example Plaintext pairs: HE LP
15     int P[2][2] = {
16         {7, 11}, // H L
17         {4, 15} // E P
18     };
19
20     // Corresponding Ciphertext pairs: TC FX
21     int C[2][2] = {
22         {19, 5}, // T F
```

Recovered Key Matrix:

15 24
22 1

=== Code Execution Successful ===



main.c



Share

Run

Output

Clear

```
48- for (int j = 1 + 1; j < 26; j++) {
49-     if (results[j].score > results[i].score) {
50-         Result temp = results[i];
51-         results[i] = results[j];
52-         results[j] = temp;
53-     }
54- }
55- }
56-
57- if (topN > 26)
58-     topN = 26;
59-
60- printf("\nTop %d Possible Plaintexts:\n", topN);
61-
62- for (int i = 0; i < topN; i++) {
63-     printf("\nKey = %d\n", results[i].key);
64-     printf("Plaintext = %s\n", results[i].text);
65-     printf("Score = %d\n", results[i].score);
66- }
67-
68- return 0;
69- }
```

```
Enter Ciphertext: KH00R ZRU0G
Enter number of top plaintexts to display: 10

Top 10 Possible Plaintexts:

Key = 10
Plaintext = AXEEH PHKEW
Score = 3

Key = 13
Plaintext = XUBBE MEHBT
Score = 2

Key = 6
Plaintext = EBIIIL TLOIA
Score = 1

Key = 2
Plaintext = IFMMP XPSME
Score = 1

Key = 3
```



main.c



Output

```
42
43 // Decrypt using frequency mapping
44 for(int i = 0; cipher[i] != '\0'; i++) {
45     if(isalpha(cipher[i])) {
46         if(isupper(cipher[i]))
47             plain[i] = map[cipher[i] - 'A'];
48         else
49             plain[i] = tolower(map[toupper(cipher[i]) - 'A']);
50     } else {
51         plain[i] = cipher[i];
52     }
53 }
54 plain[strlen(cipher)] = '\0';
55
56 // Display frequencies
57 printf("\nLetter Frequencies:\n");
58
```

Enter Cipher Text:
WKLV LV D WHVW PHVVDJH

Letter Frequencies:
A : 0
B : 0
C : 0
D : 2
E : 0
F : 0
G : 0
H : 3
I : 0
J : 1
K : 1
L : 2

main.c

Share

Run

```
12     };
13
14     printf("Encryption Round Keys:\n");
15     for(i = 0; i < 16; i++) {
16         printf("%s ", keys[i]);
17     }
18
19     printf("\n\nDecryption Round Keys (Reverse Order):\n");
20
21     // DES decryption uses keys in reverse order
22     for(i = 15; i >= 0; i--) {
23         printf("%s ", keys[i]);
24     }
25
26     return 0;
27 }
```

Output

Encryption Round Keys:
K1 K2 K3 K4 K5 K6 K7 K8 K9 K10 K11 K12 K13 K14 K15 K16

Decryption Round Keys (Reverse Order):
K16 K15 K14 K13 K12 K11 K10 K9 K8 K7 K6 K5 K4 K3 K2 K1

=== Code Execution Successful ===

main.c				Share	Run	Output
25	<pre>printf("Second 24 bits of Subkey are selected from D.\n");</pre>					Initial 56-bit Key : 111100001100110010101010111101010101100110011110001111 Left Half (C - 28 bits): 1111000011001100101010101111 Right Half (D - 28 bits): 0101010101100110011110001111 First 24 bits of Subkey are selected from C. Second 24 bits of Subkey are selected from D. Subkey (48 bits): 111100001100110010101010010101010110011001111000 === Code Execution Successful ===
26						
27	<pre>printf("\nSubkey (48 bits):\n");</pre>					
28						
29	<pre>// First 24 bits from C</pre>					
30	<pre>for(i = 0; i < 24; i++)</pre>					
31	<pre> printf("%c", C[i]);</pre>					
32						
33	<pre>// Next 24 bits from D</pre>					
34	<pre>for(i = 0; i < 24; i++)</pre>					
35	<pre> printf("%c", D[i]);</pre>					
36						
37	<pre>printf("\n");</pre>					
38						
39	<pre>return 0;</pre>					
40	<pre>}</pre>					



main.c

 Share

Run

```
24
25 // CBC Step: Plaintext XOR IV
26 for(i = 0; i < 8; i++)
27     cipher[i] = plaintext[i] ^ iv[i];
28
29 // Simulated 3DES encryption
30 encrypt(cipher, key);
31
32 printf("\nCiphertext (Hex): ");
33 for(i = 0; i < 8; i++)
34     printf("%02X ", (unsigned char)cipher[i]);
35
36 printf("\n");
37
38 return 0;
```

Output

Enter 8-character plaintext: COMPUTER
Enter 8-character key: SECURITY
Enter 8-character IV: 12345678

Ciphertext (Hex): 21 38 3D 31 32 2B 26 33

=== Code Execution Successful ===

main.c



Share

Run

Output

```
18
19 printf("Enter 8-character key: ");
20 scanf("%8s", key);
21
22 strcpy(ciphertext, plaintext);
23
24 encrypt(ciphertext, key);
25
26 printf("\nCiphertext (Hex): ");
27 for(i = 0; i < 8; i++)
28     printf("%02X ", (unsigned char)ciphertext[i]);
29
30 printf("\n");
31
32 return 0;
```

Enter 8-character plaintext: SECURITY
Enter 8-character key: NETWORK

Ciphertext (Hex): 1D 00 17 02 1D 1B 1F 59

=== Code Execution Successful ===